Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

"Linux Process Tree Visualizer"

A C++17-Based Linux Process Monitoring and Hierarchy Visualization Tool

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

J. AMRUTHA VARSHINI 2520080066

HASITH VARMA 2520090125

G. SAI KEERTHANA 2520090132

Batch No: 4

Under the Esteemed Guidance of

[Raghupathi M]
Department of [CSIT]

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Declaration

The Project Report entitled **"Linux Process Tree Visualizer"** is a record of Bonafide work of **AMRUTHA VARSHINI - 2520080066, HASITH - 2520090125, KEERTHANA - 2520090132**, Batch No: 4, submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

AMRUTHA VARSHINI-2520080066

HASITH-2520090125

KEERTHANA-2520090132

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHONOLOGY

CERTIFICATE

This is to certify that the mini project based report entitled **"Linux Process Tree Visualizer"** is a bonafide work done and submitted by **AMRUTHA VARSHINI - 2520080066, HASITH - 2520090125, KEERTHANA - 2520090132**, Batch No: 4, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science And Information Technology, K L (Deemed to be University), during the academic year **[2025-2029]**.

Signature of the Guide	**Signature of the Course Coordinator**	**Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation goes to our Course Coordinator **[Dr. K. Anusha]** and our guide **[Raghupathi.M]**, Department of [CSIT], for their tremendous support, encouragement and guidance for this project.

We express our gratitude to**Dr. Khaja Shareef (CSIT)**, Head of the Department of Computer Science And Information Technology , for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff, and all who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

AMRUTHA VARSHINI-2520080066

HASITH-2520090125

KEERTHANA-2520090132

Abstract

The **Linux Process Tree Visualizer** is an operating systems and data structures project written in **C++17** that reads live process information from the Linux **/proc** filesystem and presents it as a hierarchical tree in the terminal. On a Linux system many processes run at the same time, and every process is created by another process. For beginners, finding out which process created which, using separate terminal commands, is slow and confusing. This project addresses that problem with a lightweight, read-only application.

The application discovers processes through the numeric directories of /proc and reads the file /proc/[PID]/stat of each process. It extracts the process ID (PID), parent process ID (PPID), process name, process state, user and system CPU ticks, and resident memory. The processes are stored in a map indexed by PID, and each child is grouped under its parent using a vector. A recursive function then prints the hierarchy as an indented tree. The parser handles process names that contain spaces or parentheses, and the scanner skips processes that disappear while the snapshot is being taken.

Users interact through a simple command interface. They can display the tree, search process names without case sensitivity, view the details of a given PID, see the number of processes, the number of processes in each state, and the approximate total resident memory, refresh the snapshot on demand, and exit cleanly. The tool operates in read-only mode and never terminates or modifies any process. It is built with GNU g++ and Make and is intended to run on Ubuntu Linux or Ubuntu on Windows Subsystem for Linux (WSL).

The project demonstrates important operating system concepts such as processes, PID and PPID, the process tree, process states and resident memory, together with data structures such as maps, vectors, trees and recursion. The present version uses manual refresh. Automatic refresh, CPU percentage monitoring, process history and a graphical interface are proposed as future enhancements and are not part of the current implementation.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction		7
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Objectives of the Project	
2	OS Concepts and Background Study	2.1 Operating System Concepts Related to the Project	8
		2.2 The /proc Filesystem and the stat File	
3	Technologies Used		10
4	System Architecture and Design		11
		4.1 Process Data Collector and Parser	
		4.2 Process Tree Builder	
		4.3 Search and Statistics Module	
		4.4 Terminal Visualization and Command Interface	
		4.5 Working Methodology	
		4.6 Implementation Modules	
5	Testing and Validation		14
		5.1 Screenshots of Test Execution	
6	Results		15
7	Limitations		16
8	Future Enhancements		17
9	Conclusion		19
	References		20

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Every program that runs on Linux is a **process**. A process is created by another process, its **parent**, so all processes together form a family tree. Linux publishes information about running processes through the **/proc** pseudo-filesystem. This information is generated by the kernel when it is read; it is not stored on disk.

Operating Systems courses explain process creation and parent-child relationships in theory. Students often find it hard to connect these ideas to a real running system. This project turns the raw PID and PPID numbers found in /proc into a tree that a beginner can read, in a safe, read-only way.

1.2 Problem Statement

Linux systems execute multiple processes simultaneously. Understanding parent-child process relationships, inspecting process identifiers, and locating specific processes can be difficult for beginners when using separate terminal commands. The aim is to develop a lightweight, read-only C++ application that reads real Linux process information, reconstructs the process hierarchy, and displays it as an understandable tree.

1.3 Objectives of the Project

- Discover running processes from the Linux /proc filesystem.
- Read the PID, PPID, name, state, CPU ticks and resident memory of each process.
- Reconstruct parent-child relationships.
- Display the process hierarchy as an indented terminal tree.
- Search process names without case sensitivity.
- Display the details of a process using its PID.
- Show process counts, state counts and approximate total resident memory.
- Refresh the process snapshot on request and tolerate processes that disappear during scanning.
- Remain read-only, without killing or modifying any process.
- Demonstrate operating-system and data-structure concepts.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

- **Process:** a running instance of a program with its own process ID.
- **PID:** the Process ID, a number that identifies a process.
- **PPID:** the Parent Process ID, the PID of the process that created it.
- **Parent-child relationship:** processes form a tree because each process has one parent.
- **Process states:** show what a process is currently doing (see the table below).
- **Resident Set Size (RSS):** the number of memory pages of a process currently in RAM, multiplied by the system page size.
- **Snapshot:** the process list at one moment. It is not atomic, so processes may start or exit while it is being read.

State	Meaning
R	Running or ready to run
S	Sleeping (waiting for an event)
D	Uninterruptible sleep, usually waiting for I/O
T	Stopped
Z	Zombie: finished, but the parent has not collected its status
I	Idle kernel thread

Table 2.1: Common process state codes

2.2 The /proc Filesystem and the stat File

Each running process has a directory /proc/[PID]/. The file **/proc/[PID]/stat** holds one line of space-separated fields. The fields used by this project are listed below. Visibility of entries depends on permissions and system settings.

Field no.	Content	Used for
1	PID	Process identity
2	Name in parentheses	Display and search
3	State	State display and counts
4	PPID	Building the tree
14	utime (user CPU ticks)	Stored; not a percentage
15	stime (system CPU ticks)	Stored; not a percentage
24	rss (resident pages)	Memory in KB

Table 2.2: Fields of /proc/[PID]/stat used by the project

The process name can contain spaces and even parentheses, so the line cannot simply be split on spaces. The name is taken as the text between the **first "("** and the **last ")"**, and the remaining fields are read after it. Resident memory is calculated as resident pages multiplied by the page size and shown in KB. Summing the memory of all processes can count shared memory more than once, so the total is only approximate. CPU tick counters are cumulative values and are not the same as CPU percentage.

CHAPTER -3 TECHNOLOGIES USED

- **Programming language:** C++17
- **Compiler:** GNU g++
- **Build automation:** Make
- **Standard library:** filesystem access, maps, vectors, strings and input/output
- **Data source:** Linux /proc filesystem
- **Operating system:** Ubuntu Linux / Ubuntu on Windows Subsystem for Linux (WSL)
- **Version control:** Git and GitHub [Add repository URL]

The application can be developed and executed in Ubuntu on Windows using WSL. Windows PowerShell alone is not the target environment because /proc exists only on Linux.

CHAPTER -4 SYSTEM ARCHITECTURE AND DESIGN

The system consists of four modules arranged as a pipeline. Data flows from the Linux /proc filesystem to the terminal.

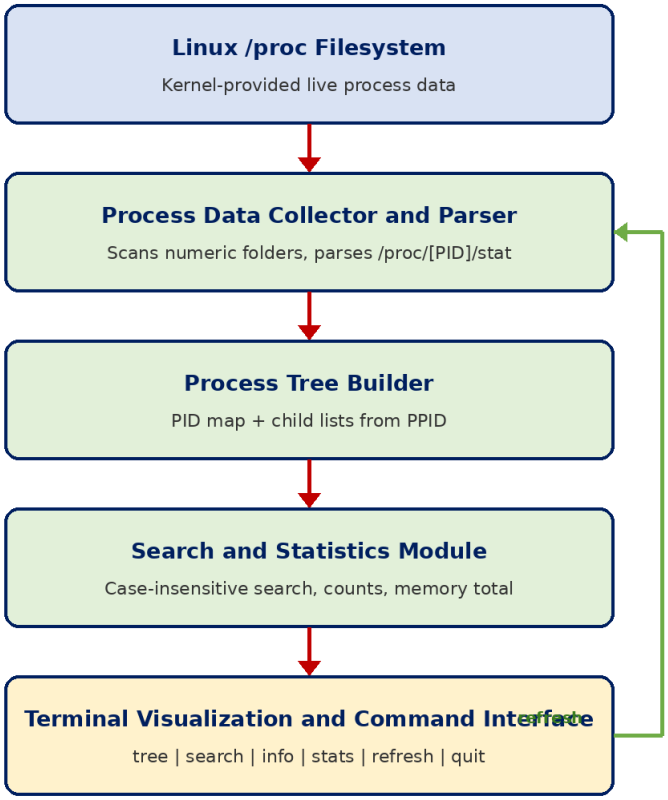


Figure 4.1: System Architecture of the Linux Process Tree Visualizer

4.1 Process Data Collector and Parser

This module iterates over the entries of /proc and keeps only the names made entirely of digits. It opens /proc/[PID]/stat, extracts the PID, name, state, PPID, user ticks, system ticks and resident pages, and skips any process that cannot be read because it has exited or is not accessible.

4.2 Process Tree Builder

The parsed records are stored in a **map** whose key is the PID. For every record, its PID is added to a **vector** of children kept under its parent PID. A process whose parent is not present in the snapshot is shown at the top level. This does not always mean it is a strict orphan; the parent may have exited or be unreadable.

4.3 Search and Statistics Module

Search converts the query and each process name to lower case and checks whether the name contains the query. The statistics function makes one pass over all processes to count the total, count each state and add the resident memory.

4.4 Terminal Visualization and Command Interface

The interface reads commands from the user and prints the results. The tree is printed by a recursive function that indents each level.

Command	Purpose
tree	Display the process hierarchy
search <text>	List processes whose names contain the text (case-insensitive)
info <pid>	Show details of one process
stats	Show process count, state counts and approximate resident memory
refresh	Take a new snapshot of /proc
quit	Exit the program

Table 4.1: Application commands

```
system [PID 1]
├── service [PID 200]
│   ├── worker [PID 240]
│   └── worker [PID 241]
└── shell [PID 300]
```

Figure 4.2: Illustrative process tree (example only, not captured output)

[Insert screenshot here]

Screenshot of the terminal showing the application prompt and the tree command

Figure 4.3: Terminal command interface

4.5 Working Methodology

1. The user starts the application in an Ubuntu or WSL terminal.
2. The collector scans the numeric directories in /proc.
3. The stat file of each process is read and parsed.
4. PID, PPID, name, state, ticks and resident memory are extracted.
5. Processes are stored in a map indexed by PID.
6. Each child process is grouped under its parent.
7. The hierarchy is traversed recursively and printed.
8. The user can search, inspect a PID, view statistics or refresh the snapshot.
9. The user exits with the quit command.

4.6 Implementation Modules

File	Responsibility
include/process_tree.hpp	Declares the process record and the collector, tree, search and statistics interface
src/process_tree.cpp	Scanning, parsing, tree building, search, statistics and tree printing
src/main.cpp	Command loop and user interaction
Makefile	Builds the executable process-visualizer with g++ in C++17 mode

Table 4.2: Project files and responsibilities

CHAPTER -5 TESTING AND VALIDATION

The following manual tests are planned. A test is marked Pass or Fail only after it has been run and evidence has been saved. The detailed procedure is given in the file TESTING.md of the repository.

ID	Test	Expected result	Actual result	Status
T01	Application startup	Program starts and shows a prompt	[Add]	Not Run
T02	Process discovery	Process count shown and close to ps output	[Add]	Not Run
T03	Tree structure	Children appear under their parent	[Add]	Not Run
T04	Process-name search	Matches found in any letter case	[Add]	Not Run
T05	Search with no matches	No-match message, no crash	[Add]	Not Run
T06	Valid PID lookup	Details of the process shown	[Add]	Not Run
T07	Invalid PID input	Error message, program continues	[Add]	Not Run
T08	Process-state display	States match ps for the same processes	[Add]	Not Run
T09	Memory statistics	Resident memory close to VmRSS; total labelled approximate	[Add]	Not Run
T10	Refresh	A new snapshot is taken	[Add]	Not Run
T11	Process creation and termination	Test process appears, then disappears after refresh	[Add]	Not Run
T12	Missing or inaccessible /proc entries	No crash; entries skipped	[Add]	Not Run
T13	Graceful exit	Program exits cleanly	[Add]	Not Run
T14	Clean build	make succeeds from a clean checkout	[Add]	Not Run

Table 5.1: Test plan (no test has been run at the time of writing)

Demonstration scenario: start a harmless test process with "sleep 120 &" and note its PID. In the visualizer use search sleep, info <PID> and tree. Stop only that process with "kill <PID>", then use refresh and search sleep again to confirm it has disappeared.

CHAPTER -6 RESULTS

The implemented baseline provides the features listed below. The test results and observations will be recorded after the tests of Chapter 5 have been executed. No result is stated here that has not been observed.

Feature	Implemented in baseline
Process discovery from /proc	Yes
PID, PPID, name, state, CPU ticks, resident memory	Yes
Indented process tree	Yes
Case-insensitive name search	Yes
Process details by PID	Yes
Process, state and approximate memory statistics	Yes
Manual refresh	Yes
Handling of vanished processes	Yes
Read-only operation	Yes
Automatic refresh, CPU percentage, history, GUI	No (future work)

Table 6.1: Feature status

Observations:

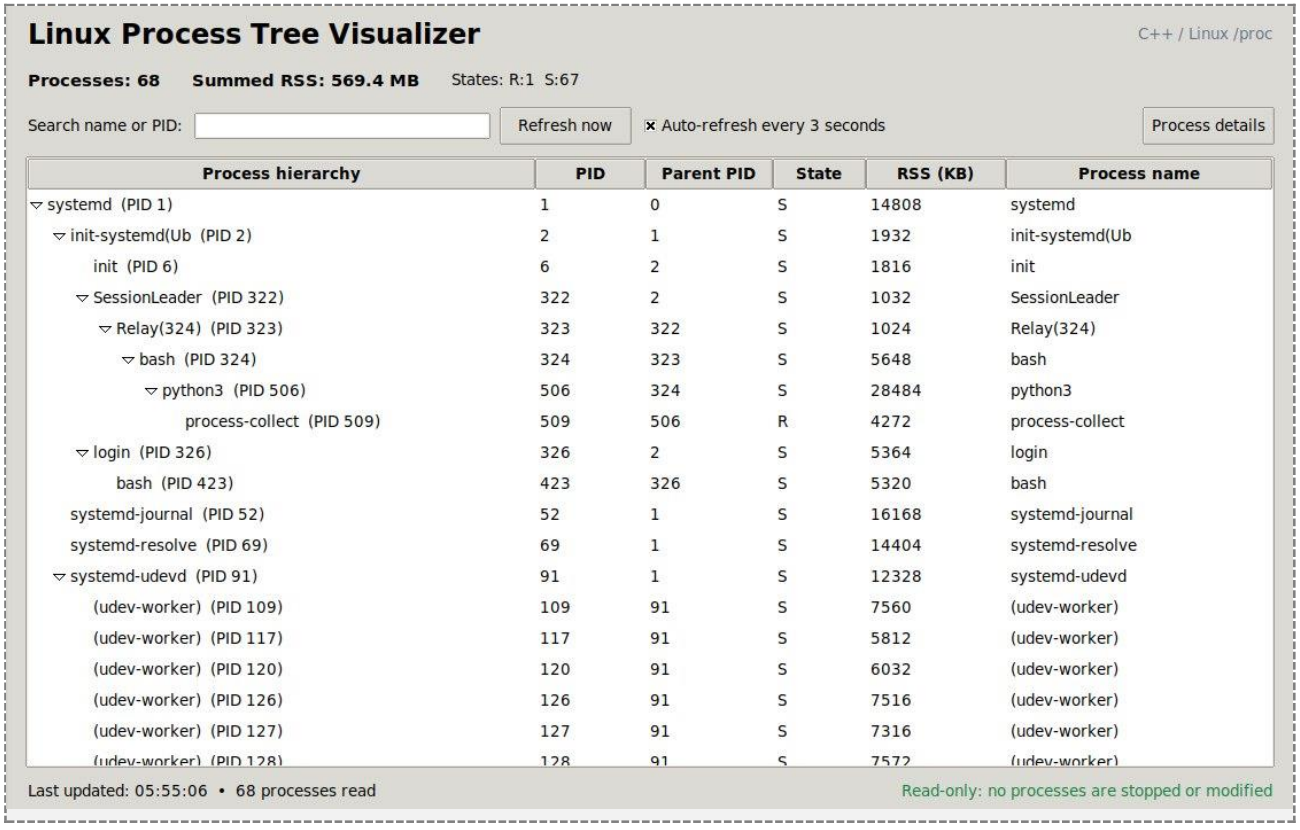


Figure 6.1: Application output

CHAPTER -7 LIMITATIONS

- The current version uses manual refresh only.
- Snapshots are not atomic; processes may start or exit while /proc is being read.
- The application works only on Linux, Ubuntu or WSL, because it needs /proc.
- Some /proc entries may be unreadable depending on permissions and visibility.
- The summed resident memory is approximate because shared memory may be counted more than once.
- CPU values are raw tick counters, not CPU percentages.
- A missing parent in the snapshot does not always mean a strict orphan process.
- The interface is terminal-based; there is no graphical interface.

CHAPTER -8 FUTURE ENHANCEMENTS

The following are proposed and are not implemented in the current version.

- Automatic periodic refresh.
- CPU percentage calculation using two snapshots.
- Process creation and termination history.
- Highlighting of resource-intensive processes.
- An interactive graphical interface.
- Export of the tree to a file and additional automated tests.;

CHAPTER -8 FUTURE ENHANCEMENTS

The following are proposed and are not implemented in the current version.

- Automatic periodic refresh.
- CPU percentage calculation using two snapshots.
- Process creation and termination history.
- Highlighting of resource-intensive processes.
- An interactive graphical interface.
- Export of the tree to a file and additional automated tests.;

CHAPTER -9 CONCLUSION

The Linux Process Tree Visualizer shows how data from the Linux /proc filesystem can be converted into a readable process hierarchy. By combining a map for fast lookup by PID, vectors for child lists and a recursive traversal, the project connects data structures with operating-system concepts such as PID, PPID, process states and resident memory. The tool is read-only, simple to build with g++ and Make, and suitable for learning on Ubuntu or WSL. Automatic refresh, CPU percentage, process history and a graphical interface can be added in future versions.

REFERENCES

1. Linux manual pages: proc(5), the /proc filesystem (man 5 proc).
2. Linux manual pages: fork(2), sysconf(3).
3. ISO/IEC 14882:2017, Programming Languages - C++ (C++17), and cppreference.com.
4. GNU Make Manual, Free Software Foundation.
5. [Add course textbook and any other sources actually used]